Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №4
по курсу «Технологии глубоких нейронных сетей»
на тему «Создание нейронной сети PyTorch»

Выполнил:
студент группы 23ВВИ2
Юров Д.М.

Принял:
Митрохин М.А.

Пенза 2026

Цель работы: изучить основы создания и обучения нейронных сетей в PyTorch на примере задач классификации и регрессии. Получить практические навыки подготовки признаков и построения модели для прогнозирования и распознавания классов.

Задание:

1. Подготовка
   1) Откройте Anaconda Prompt или консоль команд, активируйте созданную для выполнения лабораторных работ среду с установленным PyTorch.
   2) Запустите IDE,
   3) Откройте файл «Lab4_pytorch_net.py»,
   4) Прочитайте комментарии и выполните код,
   5) После ознакомления с кодом выполните задание.
2. Задание 1
   1) Загрузите данные из файла «dataset_simple.csv»,
   2) На основе имеющихся примеров кода создайте нейронную сеть, решающую задачу по варианту.

Вариант 1: решите задачу классификации покупателей на классы «купит»/«не купит» (3 столбец) по признакам возраст и доход.

Вариант 2: решите задачу предсказания дохода по возрасту.

Ход выполнения:

1. Подготовка

1) Открыл Anaconda Prompt, активировал среду «dnnenv» с установленным PyTorch (рис. 1).

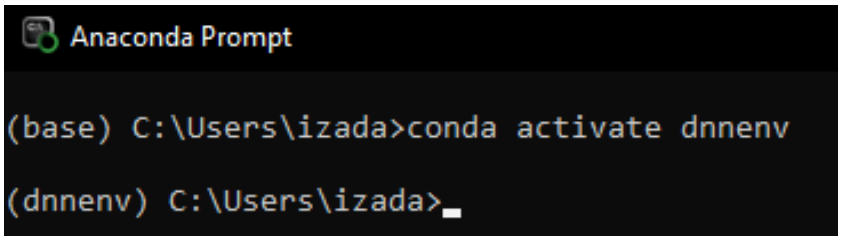

Рисунок 1 - "Anaconda Prompt" с активированной средой

2) Запустил IDE Spyder (рис. 2).

Рисунок 2 - запуск "Spyder"

3) Открыл файл «Lab4_pytorch_net.py», прочитал комментарии (рис. 3).

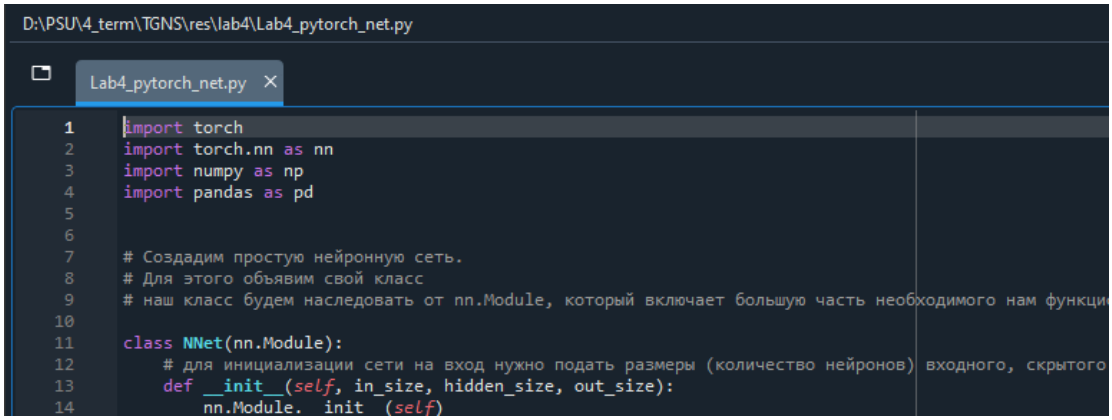

Рисунок 3 - файл "Lab4_pytorch_net.py"

4) Выполнил код (рис. 4).

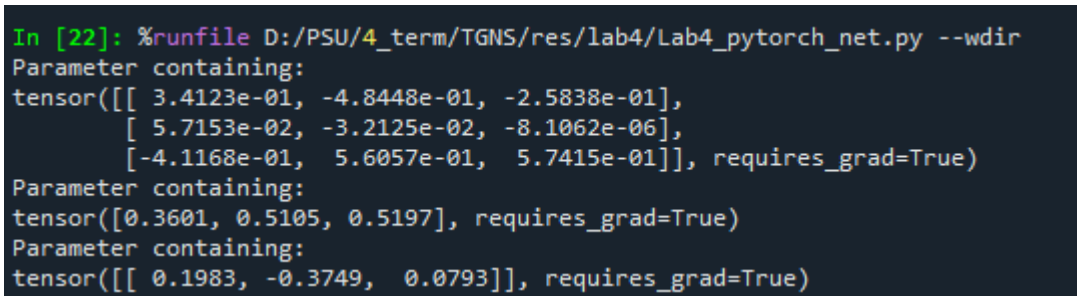

Рисунок 4 - результат выполнения кода "Lab4_pytorch_net.py"

2. Задание 1

1) На основе имеющихся примеров кода создал нейронную сеть, решающую задачу классификации покупателей на классы «купит»/«не купит» (см. Приложение 1 – файл «res1.py»).

2) Выполнил код созданной программы (рис. 5).

```
In [78]: %runfile d:/psu/4_term/tgns/res/lab4/res1.py --wdir
hidden_size =  3
ошибки:  tensor([67.], device='cuda:0')
*обчение*
ошибка на  0 операции:  1.723905324935913
ошибка на  500 операции:  0.3815881609916687
ошибка на  1000 операции:  0.2375289648771286
ошибка на  1500 операции:  0.16954568028450012
ошибка на  2000 операции:  0.13266205787658691
ошибка на  2500 операции:  0.11026599258184433
ошибка на  3000 операции:  0.09532754868268967
ошибка на  3500 операции:  0.0846288800239563
ошибка на  4000 операции:  0.0765530988574028
ошибка на  4500 операции:  0.07021280378103256
ошибка на  5000 операции:  0.06508314609527588
ошибка на  5500 операции:  0.06083399057388306
*проверка обученной сети*
ошибки: tensor([0.], device='cuda:0')
pred:  tensor([-1.,  1.,  1., -1.,  1., -1.,  1.,  1., -1.,  1., -1.,  1.,  1., -1.,
        -1.,  1.,  1.,  1.,  1.,  1., -1.,  1.,  1.,  1., -1., -1.,  1.,  1.,
         1., -1., -1.,  1., -1.,  1.,  1.,  1.,  1., -1.,  1.,  1.,  1.,  1.,
        -1.,  1.,  1.,  1., -1.,  1., -1.,  1.,  1., -1.,  1.,  1.,  1.,  1.,
        -1.,  1.,  1.,  1., -1.,  1.,  1., -1.,  1.,  1., -1.,  1.,  1.,  1.,
         1.,  1., -1., -1.,  1.,  1.,  1.,  1., -1.,  1.,  1.,  1.,  1.,  1.,
         1., -1., -1.,  1., -1.,  1.,  1.,  1.,  1.,  1., -1.,  1.],
       device='cuda:0')
y:     tensor([-1.,  1.,  1., -1.,  1., -1.,  1.,  1., -1.,  1., -1.,  1.,  1., -1.,
        -1.,  1.,  1.,  1.,  1.,  1., -1.,  1.,  1.,  1., -1., -1.,  1.,  1.,
         1., -1., -1.,  1., -1.,  1.,  1.,  1.,  1., -1.,  1.,  1.,  1.,  1.,
        -1.,  1.,  1.,  1., -1.,  1., -1.,  1.,  1., -1.,  1.,  1.,  1.,  1.,
        -1.,  1.,  1.,  1., -1.,  1.,  1., -1.,  1.,  1., -1.,  1.,  1.,  1.,
         1.,  1., -1., -1.,  1.,  1.,  1.,  1., -1.,  1.,  1.,  1.,  1.,  1.,
         1., -1., -1.,  1., -1.,  1.,  1.,  1.,  1.,  1., -1.,  1.],
       device='cuda:0')
```

Рисунок 5 - результат выполнения кода "res1.py"

3) На основе имеющихся примеров кода создал нейронную сеть, решающую задачу предсказания дохода по возрасту (см. Приложение 2 – файл «res2.py»)

4) Выполнил код созданной программы (рис. 6-7).

```
In [21]: %runfile D:/PSU/4_term/TGNS/res/lab4/res2.py --wdir

hidden_size =  3

ошибка на  0 операции:  50026.625
ошибка на  1000 операции:  4079.08740234375
ошибка на  2000 операции:  4047.578125
ошибка на  3000 операции:  4013.26904296875
ошибка на  4000 операции:  3973.55419921875
ошибка на  5000 операции:  3926.66162109375
ошибка на  6000 операции:  3872.640625
ошибка на  7000 операции:  3810.739013671875
ошибка на  8000 операции:  3744.23193359375
ошибка на  9000 операции:  3684.59423828125

*проверка обученной сети*

максимальная ошибка:  tensor(4713.9492, device='cuda:0')

pred:  tensor([29830.5176, 43295.5469, 56760.5742, 36563.0312, 63493.0898, 33870.0273,
        50028.0625, 70225.5938, 39256.0391, 60800.0859, 25791.0078, 48681.5547,
        51374.5625, 35216.5312, 40602.5430, 58107.0781, 41949.0430, 44642.0508,
        59453.5781, 70225.5938, 31177.0195, 47335.0547, 52721.0664, 64839.5898,
        35216.5312, 39256.0391, 45988.5547, 67532.5938, 60800.0859, 32523.5215,
        36563.0312, 66186.0859, 37909.5352, 55414.0742, 54067.5664, 43295.5469,
        62146.5859, 31177.0195, 41949.0430, 50028.0625, 52721.0664, 64839.5898,
        37909.5352, 60800.0859, 68879.1016, 71572.1016, 35216.5312, 45988.5547,
        32523.5215, 48681.5547, 56760.5742, 33870.0273, 44642.0508, 67532.5938,
        41949.0430, 62146.5859, 36563.0312, 43295.5469, 54067.5664, 66186.0859,
        31177.0195, 47335.0547, 59453.5781, 33870.0273, 51374.5625, 55414.0742,
        37909.5352, 45988.5547, 70225.5938, 40602.5430, 58107.0781, 62146.5859,
        39256.0391, 36563.0312, 47335.0547, 43295.5469, 56760.5742, 51374.5625,
        27137.5117, 40602.5430, 60800.0859, 50028.0625, 67532.5938, 52721.0664,
        64839.5898, 35216.5312, 31177.0195, 45988.5547, 39256.0391, 41949.0430,
        63493.0898, 60800.0859, 43295.5469, 56760.5742, 37909.5352, 48681.5547],
       device='cuda:0')

y:     tensor([30000., 40000., 60000., 35000., 80000., 32000., 45000., 70000., 38000.,
        75000., 28000., 42000., 49000., 33000., 37000., 58000., 39000., 41000.,
        62000., 80000., 31000., 48000., 55000., 78000., 34000., 36000., 42000.,
        65000., 72000., 33000., 38000., 77000., 37000., 59000., 56000., 40000.,
        78000., 32000., 38000., 49000., 51000., 75000., 36000., 68000., 77000.,
        81000., 34000., 43000., 32000., 48000., 58000., 33000., 42000., 67000.,
        37000., 74000., 35000., 43000., 54000., 78000., 30000., 45000., 59000.,
        34000., 48000., 56000., 36000., 43000., 72000., 38000., 61000., 74000.,
        38000., 35000., 48000., 40000., 59000., 51000., 29000., 40000., 64000.,
        50000., 69000., 52000., 76000., 36000., 30000., 45000., 37000., 39000.,
        77000., 71000., 42000., 57000., 35000., 49000.], device='cuda:0')

In [22]:
```

Рисунок 6 - результат выполнения кода "res2.py"

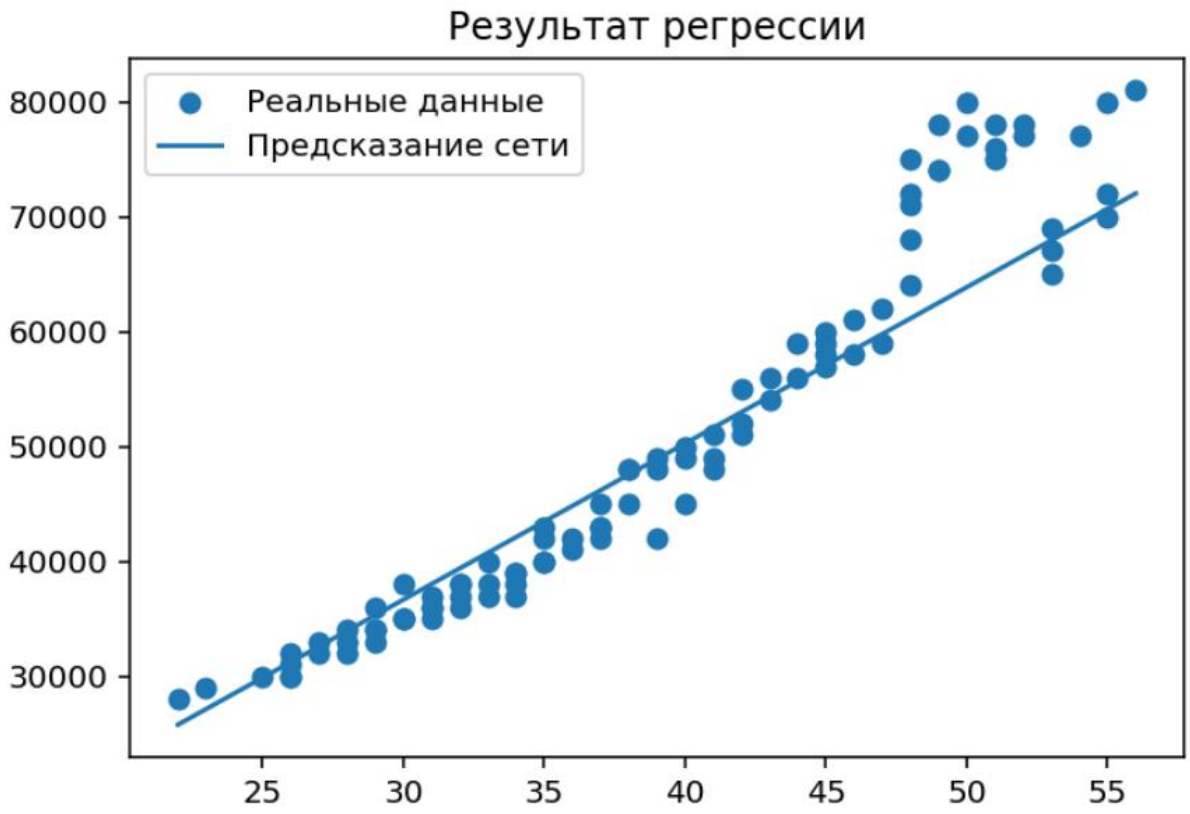

Рисунок 7 - график результатов регрессии

Вывод: изучить основы создания и обучения нейронных сетей в PyTorch на примере задач классификации и регрессии. Получить практические навыки подготовки признаков и построения модели для прогнозирования и распознавания классов.

Приложение 1 – код «res1.py»

```
# задача классификации "купит" - "не купит"

import torch
import torch.nn as nn
import pandas as pd
import numpy as np


class ClassificationNet(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        nn.Module.__init__(self)
        self.layers = nn.Sequential(nn.Linear(input_size, hidden_size),
                                    nn.Tanh(),
                                    nn.Linear(hidden_size, output_size),
                                    nn.Tanh())

    def forward(self, x):
        pred = self.layers(x)
        return pred

if(torch.cuda.is_available()):
    device = "cuda:0"
else:
    device = "cpu"



filename = "dataset_simple.csv"

df = pd.read_csv(filename)


x_np = df.iloc[:, [0, 1]].values.astype(np.float32)
x_np = (x_np - x_np.mean(axis=0)) / x_np.std(axis=0)
x = torch.Tensor(x_np).to(device)

y = df.iloc[:,[2]].values
y = torch.Tensor(np.where(y == 1, 1, -1).reshape(-1,1)).to(device)

input_size = 2
hidden_size = 3
output_size = 1
print("hidden_size = ", hidden_size)

net = ClassificationNet(input_size, hidden_size, output_size).to(device)



# проверка работы до обучения:
#print("*проверка до обучения*")
with torch.no_grad():
    pred = net.layers(x)

pred = (pred >= 0).float() * 2 - 1
#print(pred)

err = sum(abs(y-pred))/2
print("ошибки: ", err)
```

```
# обучение:
print("*обчение*")
lossFn = nn.MSELoss()

optimizer = torch.optim.SGD(net.parameters(), lr=0.005)

epochs = 6000
for i in range(0, epochs):
    pred = net.layers(x)
    loss = lossFn(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if(i % 500 == 0):
        print("ошибка на ", i, "операции: ", loss.item())

#проверка работы после обучения:
print("*проверка обученной сети*")

with torch.no_grad():
    pred = net.layers(x)

pred = (pred >= 0).float() * 2 - 1
err = sum(abs(y-pred))/2


# вывод результатов
print("ошибки:", err)
#print("предсказание - истина")
#for i in range(0, pred.shape[0]):
#    print(pred[i].item(), " - ", y[i].item())

print("pred: ", pred.squeeze())
print("y:    ", y.squeeze())
```

Приложение 2 – код «res2.py»

```
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
import torch
import torch.nn as nn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


class RegressNet(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        nn.Module.__init__(self)

        self.layers = nn.Sequential(nn.Linear(input_size, hidden_size),
                                    nn.ReLU(),
                                    nn.Linear(hidden_size, output_size)
                                    )

    def forward(self, x):
        pred = self.layers(x)
        return pred


if(torch.cuda.is_available()):
    device = "cuda:0"
else:
    device = "cpu"



filename = "dataset_simple.csv"

df = pd.read_csv(filename)

x = torch.Tensor(df.iloc[:, [0]].values).to(device)
y = torch.Tensor(df.iloc[:, [1]].values).to(device)


plt.figure()
plt.scatter(df.iloc[:, [0]].values, df.iloc[:, 1].values, marker='o')

input_size = x.shape[1]
hidden_size = 3
output_size = y.shape[1]

print("\nhidden_size = ", hidden_size, "\n")

net = RegressNet(input_size, hidden_size, output_size).to(device)

lossFn = nn.L1Loss()

optimizer = torch.optim.SGD(net.parameters(), lr=0.001)



epochs = 10000
for i in range(0, epochs):
    pred = net.layers(x)
    loss = lossFn(pred, y)
```

```
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if(i % 1000 == 0):
        print("ошибка на ", i, "операции: ", loss.item())

#проверка работы после обучения:
print("\n*проверка обученной сети*")

with torch.no_grad():
    pred = net.layers(x)


err = (abs(y.max()-pred.max()))/2

print("\nмаксимальная ошибка: ", err)

print("\npred: ", pred.squeeze())
print("\ny:    ", y.squeeze())

x_plot = df.iloc[:, 0].values
y_plot = df.iloc[:, 1].values
pred_plot = pred.detach().cpu().numpy()
sort_idx = np.argsort(x_plot)

plt.figure()
plt.scatter(x_plot, y_plot, marker='o', label='Реальные данные')
plt.plot(x_plot[sort_idx], pred_plot[sort_idx], label='Предсказание сети')
plt.legend()
plt.title("Результат регрессии")
plt.show()
```